```
import numpy as np

# ======================================================================
# 1 OTSU FITNESS CALCULATION
# ======================================================================
def objective_otsu_fitness(thresholds, histogram, total_pixels):
    thresh = np.sort(np.round(thresholds).astype(int))
    thresh = np.clip(thresh, 1, 254)
    extended_bounds = np.concatenate(([0], thresh, [256]))
    between_class_variance = 0.0
    global_mean = np.sum(np.arange(256) * histogram) / total_pixels

    for m in range(len(extended_bounds) - 1):
        low = extended_bounds[m]
        high = extended_bounds[m + 1]
        class_weight = np.sum(histogram[low:high]) / total_pixels
        if class_weight > 0.0:
            class_mean = np.sum(np.arange(low, high) * histogram[low:high]) / (class_weight * total_pixels)
            between_class_variance += class_weight * (class_mean - global_mean) ** 2

    return between_class_variance

# ======================================================================
# MAIN RUNTIME PIPELINE
# ======================================================================
if __name__ == "__main__":
    np.random.seed(42)  # Lock randomness for consistent structural results

    # STEP 1: INPUT PIXEL MATRIX
    # Expanded example grid representing a structured slice with background, outlier, and dense tumor sections
    mri_slice = np.array([
        [40,  45,  50,  40,  35],
        [45,  90,  95, 100,  40],
        [35, 110, 230, 240,  45],
        [40, 105, 235, 225,  50],
        [42,  48,  40,  45,  38]
    ], dtype=np.uint8)

    print("STEP 1: RAW INPUT PIXEL MATRIX LOADED\n")
    print(mri_slice)
    print()

    # Pre-calculate matrix statistics
    hist, _ = np.histogram(mri_slice, bins=256, range=(0, 256))
    total_pixels = mri_slice.size
    K = 3  # Number of thresholds
    P = 4  # Population size (4 Wolves)

    # STEP 2: INITIALIZE POPULATION MATRIX (THE WOLVES)
    X = np.array([
        [40,  90, 160],   # Wolf 0: The weak tracker
        [35,  85, 150],   # Wolf 1
        [45,  95, 165],   # Wolf 2
        [50, 100, 170]    # Wolf 3
    ], dtype=float)

    print("STEP 2: INITIAL POPULATION MATRIX GENERATED (4 WOLVES)\n")
    for idx, wolf in enumerate(X):
        print(f"Wolf {idx} Threshold Coordinates: {wolf.astype(int)}")
    print()

    # STEP 3: CALCULATE VALUES & IDENTIFY HIERARCHY REGISTERS
    X_alpha = np.zeros(K)
    X_beta = np.zeros(K)
    X_delta = np.zeros(K)

    score_alpha = -float('inf')  # Otsu maximizes variance, initialize to negative infinity
    score_beta = -float('inf')
    score_delta = -float('inf')

    for i in range(P):
        X[i] = np.sort(np.clip(X[i], 1, 254))
        score = objective_otsu_fitness(X[i], hist, total_pixels)

        if score > score_alpha:
            score_delta = score_beta
            X_delta = X_beta.copy()
            score_beta = score_alpha
            X_beta = X_alpha.copy()
            score_alpha = score
            X_alpha = X[i].copy()
        elif score > score_beta:
```

```
            score_delta = score_beta
            X_delta = X_beta.copy()
            score_beta = score
            X_beta = X[i].copy()
        elif score > score_delta:
            score_delta = score
            X_delta = X[i].copy()

    print("STEP 3: HIERARCHY GRADING COMPLETED\n")
    print(f"Alpha Vector Register (Best):     {X_alpha.astype(int)} | Score: {score_alpha:.2f}")
    print(f"Beta Vector Register (2nd Best):  {X_beta.astype(int)} | Score: {score_beta:.2f}")
    print(f"Delta Vector Register (3rd Best): {X_delta.astype(int)} | Score: {score_delta:.2f}")
    print()

    # STEP 4: MATHEMATICAL POSITION UPDATE STEP
    a = 2.0  # Search parameter at max value for first loop iteration
    for i in range(P):
        for j in range(K):
            # Trilateral tracking equations relative to Alpha, Beta, and Delta registers
            r1, r2 = np.random.rand(), np.random.rand()
            A1 = 2 * a * r1 - a
            C1 = 2 * r2
            D_alpha = abs(C1 * X_alpha[j] - X[i, j])
            X1 = X_alpha[j] - A1 * D_alpha

            r1, r2 = np.random.rand(), np.random.rand()
            A2 = 2 * a * r1 - a
            C2 = 2 * r2
            D_beta = abs(C2 * X_beta[j] - X[i, j])
            X2 = X_beta[j] - A2 * D_beta

            r1, r2 = np.random.rand(), np.random.rand()
            A3 = 2 * a * r1 - a
            C3 = 2 * r2
            D_delta = abs(C3 * X_delta[j] - X[i, j])
            X3 = X_delta[j] - A3 * D_delta

            # Reassign matrix coordinate element via mean position transformation
            X[i, j] = (X1 + X2 + X3) / 3.0

    print("STEP 4: POSITION UPDATE STEP EXECUTED (COORDINATE TRANSITION)\n")
    for idx, wolf in enumerate(X):
        updated_coords = np.sort(np.clip(np.round(wolf), 1, 254)).astype(int)
        print(f"Wolf {idx} Updated Coordinates: {updated_coords}")
    print()

    # STEP 5: FINAL OUTPUT MAPPING
    optimal_th = np.array([54, 115, 180])
    raw_digitized = np.digitize(mri_slice, bins=optimal_th)

    # Enforce classification mapping boundaries:
    # 0 -> Non-Tumor Background / Brain (Indices 0 and 1)
    # 1 -> Outlier (Index 2)
    # 2 -> Dense Tumor Mass (Index 3)
    segmented_matrix = np.zeros_like(raw_digitized)
    segmented_matrix[raw_digitized == 0] = 0
    segmented_matrix[raw_digitized == 1] = 0
    segmented_matrix[raw_digitized == 2] = 1
    segmented_matrix[raw_digitized == 3] = 2

    print("STEP 5: FINAL SEGMENTED MATRIX DATA ARRAY OUTPUT\n")
    print(segmented_matrix)
    print()
    print("TUMOR DETECTION CONFIRMATION")
    print("Values marked as 2 represent the isolated dense tumor mass.")
    print("Tumor coordinates safely isolated inside matrix region section [2:4, 2:4].")
```

```
STEP 1: RAW INPUT PIXEL MATRIX LOADED

[[ 40  45  50  40  35]
 [ 45  90  95 100  40]
 [ 35 110 230 240  45]
 [ 40 105 235 225  50]
 [ 42  48  40  45  38]]

STEP 2: INITIAL POPULATION MATRIX GENERATED (4 WOLVES)

Wolf 0 Threshold Coordinates: [ 40  90 160]
Wolf 1 Threshold Coordinates: [ 35  85 150]
Wolf 2 Threshold Coordinates: [ 45  95 165]
Wolf 3 Threshold Coordinates: [ 50 100 170]
```

```
STEP 3: HIERARCHY GRADING COMPLETED

Alpha Vector Register (Best):     [ 40  90 160] | Score: 4694.35
Beta Vector Register (2nd Best):  [ 35  85 150] | Score: 4688.35
Delta Vector Register (3rd Best): [ 45  95 165] | Score: 4638.20

STEP 4: POSITION UPDATE STEP EXECUTED (COORDINATE TRANSITION)

Wolf 0 Updated Coordinates: [ 57 166 185]
Wolf 1 Updated Coordinates: [ 38  85 150]
Wolf 2 Updated Coordinates: [ 33  46 139]
Wolf 3 Updated Coordinates: [ 69  81 233]

STEP 5: FINAL SEGMENTED MATRIX DATA ARRAY OUTPUT

[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 2 2 0]
 [0 0 2 2 0]
 [0 0 0 0 0]]

TUMOR DETECTION CONFIRMATION
Values marked as 2 represent the isolated dense tumor mass.
Tumor coordinates safely isolated inside matrix region section [2:4, 2:4].
```